



How Do Priority Queues and Heaps Work?

A **priority queue** is an abstract data type (ADT) that works similarly to a queue or stack, but with one key difference.

As you may already know, standard queues follow a FIFO (First-in, First-out) approach, where the first element added to the queue is the first one to be removed from the queue.

Stacks follow a LIFO (Last-in, First-out) approach, where the last element added to the stack is the first one to be removed from the stack.

Queues and stacks only consider the order of insertion of the elements.

However, priority queues take the "priority" of the elements into account. The priority is used to determine which element should be removed next.

Usually, the element with the highest priority is removed first, but some implementations may also choose to remove the element with the lowest priority first. This will depend on the requirements of your program.

Priority queues are very helpful for practical applications like finding the shortest path between two locations, scheduling tasks in operating systems, simulating traffic, compressing data, and managing networks.

In practice, priority queues are commonly implemented using a heap data structure.

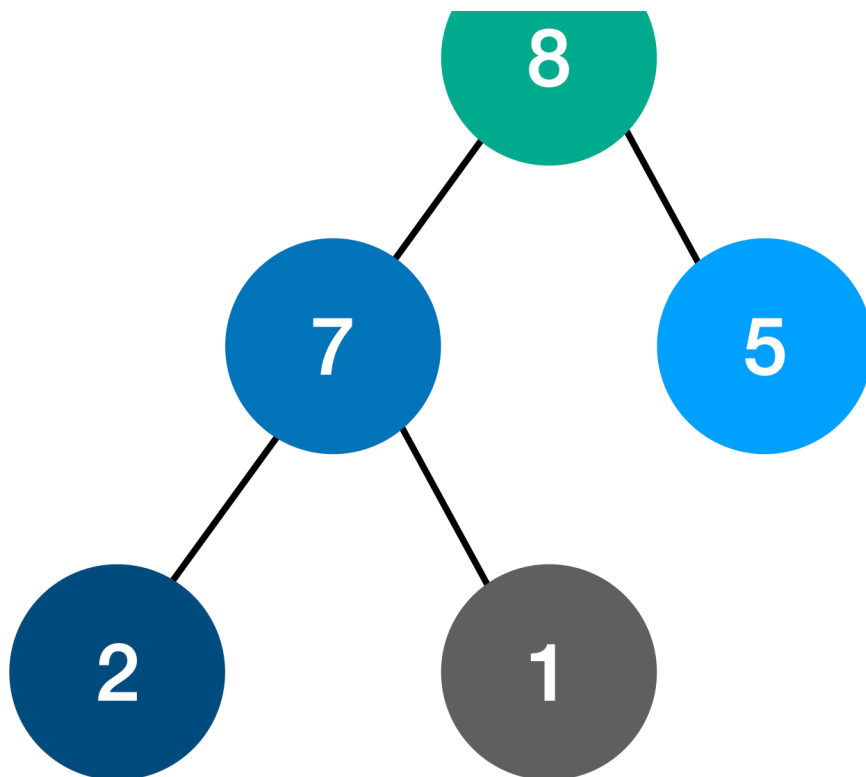
A **heap** is a tree data structure with a very specific property called the **heap property**. This property determines the relationship between each node and its children, based on the type of heap.

There are two primary types of heaps:

- Max-heap
- Min-heap

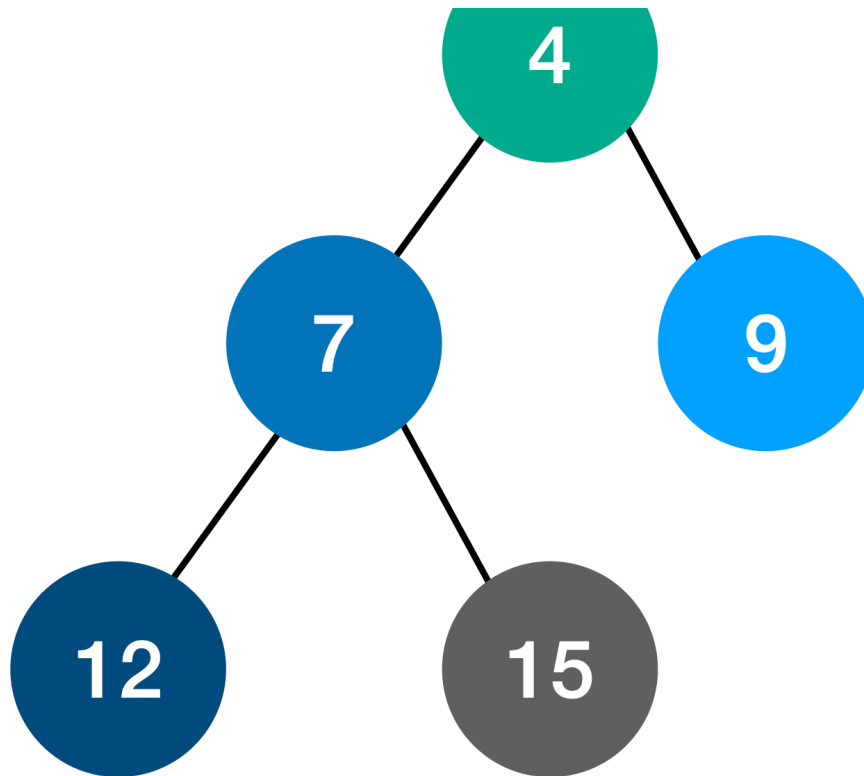
In a **max-heap**, the value of each node is greater than or equal to the value of its children.

In this example, you can see a tree structure with the nodes 8, 7, 5, 2, and 1. Note that node 7 is greater than node 2 and node 1, following the heap property. This is true for all the other nodes as well.



In contrast, in a **min-heap**, the value of each node is less than or equal to the value of its children.

In this example, we have nodes with values 4, 7, 9, 12, and 15. For example, node 7 is less than node 12 and node 15, following the heap property. This is also true for all the other nodes.



The heap property is key because it ensures that the maximum (or minimum) element (depending on the type of heap) always stays at the top, which makes it very easy to remove.

In practice, heaps are typically implemented as arrays to access parent and child nodes efficiently.

Using arrays simplifies the logic for accessing these values or "nodes" because behind the scenes, if the heap maintains the structure of a complete binary tree, the array implementation only requires simple mathematical operations based on their indices to find where the elements are located in memory.

Python has a `heapq` built-in module that you can use to work with an implementation of a min-heap.

It works by operating directly on Python lists, following specific steps that work with the elements as if the list was a heap, preserving the heap property.

To use this module, you just need to import it:

```
import heapq
```

Then, you need to define an empty list. This will be the underlying data structure for the heap:

```
my_heap = []
```

To add elements to the heap, you would call `heappush()`, passing the name of the heap and the element that you want to add as arguments. This will automatically add the element to the list where it should be, to preserve the heap property:

```
heapq.heappush(my_heap, 9)
```



```
heapq.heappop(my_heap)
```

`heappushpop()` combines both operations into one call.

This is more efficient than calling them in a sequence separately, especially when the heap is large, since it only performs one "heapify" operation to reorder the list as a heap:

```
heapq.heappushpop(my_heap, 15)
```

If you already have a list and you want to transform it into a heap, you could call `heapify()`, passing the list as argument:

```
heapq.heapify(my_heap)
```

But currently, we are sorting the elements by their values, right?

What if we want to sort them by their "priority" instead?

You could do this by storing tuples with this structure: `(priority, element)`.

Since tuples are compared element by element from left to right, the priorities will be compared first, and the decisions will be made based on them.

Please note that, in this case, lower values will represent higher priorities. This means that a tuple with priority of 1 will have a **higher** priority than a tuple with priority of 3:

```
my_heap = []

heapq.heappush(my_heap, (3, "A"))
heapq.heappush(my_heap, (2, "B"))
heapq.heappush(my_heap, (1, "C"))
```

If you need elements with the same priority to be removed in the order that they were inserted, you could consider including a unique counter as the second element of your tuple to break the tie, like this `(priority, counter, element)`.

Now let's talk about the efficiency of heaps.

The average and worst case time complexities for inserting and extracting the minimum or maximum value from a heap (depending on the type of heap) are logarithmic, `O(log n)`, because the number of swaps required is usually proportional to the height of the heap, which is $\log(n)$.

The average and worst case time complexity for the "peek" operation is constant time, `O(1)`. Peeking involves getting the minimum or maximum value (depending on the type of heap) without removing it.

The "heapify" operation, where the heap is built from an unsorted list, has linear time complexity, `O(n)`, in the average and worst cases.

Similarly, both searching for and deleting an arbitrary element have linear average and worst case time complexities of `O(n)`, since they potentially require traversing the entire heap.

And how much space do they require?

The space complexity of the heap is linear, `O(n)`, where `n` is the number of elements it contains. It only needs to store the elements and a small additional overhead for the list object itself.



Questions

What is the primary characteristic that distinguishes a priority queue from a standard queue or stack?

It allows elements to be accessed by their index.

It always processes elements in the order they were inserted.

It retrieves elements based on an assigned priority.

Correct!

It only stores elements of the same data type.

Which of the following is a common real-world application where a priority queue would be helpful?

Scheduling tasks in an operating system where some tasks are more urgent.

Correct!

Managing a playlist where songs play in a fixed order.

Storing a list of grocery items.

Keeping track of customer names in alphabetical order.

What is the main reason why heaps are typically implemented as arrays in practice, despite being conceptualized as trees?

Arrays are always faster than any other data structure.

Arrays simplify the logic for accessing parent and child nodes using mathematical formulas.

Correct!

Arrays allow for direct random access to any element, which is a core heap operation.



Python Certification



Understanding Graphs and Trees

Check your answer

Ask for Help